Spectral Shape Optimization over Polygons

The mathematics `douse` implements, and where it bites

douse development notes

2026-08-24

Abstract

We optimize functionals of the Dirichlet Laplacian spectrum over N-gons of fixed perimeter — principally λ_k/λ_1, the polygon analogue of Osting's optimization over Fourier-boundary domains. This note records the mathematics actually used: the parametrization and its Jacobian, the Hadamard shape derivative, what replaces it at a multiple eigenvalue, and the two trust-region subproblems that result. Several of the subtleties below were found by running the code and getting plausible wrong answers, and those are flagged where they arise; they are properties of the mathematics, not of the implementation.

1 The problem

For a bounded, connected, open $\Omega \subset \mathbb{R}^2$ let $0 < \lambda_1(\Omega) < \lambda_2(\Omega) \le \lambda_3(\Omega) \le \cdots$ be the Dirichlet eigenvalues,

$$-\Delta u_j = \lambda_j u_j \ \text{ in } \Omega, \qquad u_j = 0 \ \text{ on } \partial\Omega,$$

repeated by multiplicity, with $\{u_j\}$ orthonormal in $L^2(\Omega)$. Two classical facts frame everything that follows.

Fact 1 (Krahn)**.** λ_1 is simple for connected Ω: its eigenfunction may be taken positive, and two orthogonal positive functions cannot coexist.

This is why objectives built on λ_1 alone are smooth, and why the denominator of λ_k/λ_1 never participates in a degeneracy.

Fact 2 (Payne–Pólya–Weinberger; proved by Ashbaugh–Benguria)**.** For every bounded connected planar Ω,

$$\frac{\lambda_2}{\lambda_1} \ \le \ \left(\frac{j_{1,1}}{j_{0,1}}\right)^2 = 2.5387340\ldots,$$

with equality only for the disk.

Fact 2 is not merely context: we use it as a *runtime invariant*. Any reported λ_2/λ_1 above it is an error, whatever it looks like locally, and it has twice caught bugs that testing did not.

1.1 The objective

We maximize

$$F(\Omega) \;=\; \frac{\lambda_k(\Omega)}{\lambda_1(\Omega)}. \tag{1}$$

Scaling $\Omega \mapsto c\,\Omega$ sends $\lambda_j \mapsto c^{-2}\lambda_j$, so F is *scale invariant*. Consequently the perimeter constraint imposed below is pure gauge: it removes the scaling degree of freedom and imposes no physics on (1). (For an objective such as $\min \lambda_1$, which is *not* scale invariant, the same constraint is the entire content of the problem.)

2 Parametrizing N-gons

An N-gon has $2N$ coordinates, but the spectrum does not see all of them: translations, rotations and reflections leave it fixed, and scaling acts as above. Optimizing over all $2N$ would wander a four-dimensional manifold of identical spectra. We therefore fix the gauge by construction. Writing vertices as complex numbers $v_0, \dots, v_{N-1}$,

$$v_{N-1} = 0, \qquad v_0 \in \mathbb{R}_{>0}, \qquad \mathrm{per}(\Omega) = P \text{ fixed},$$

leaving $v_1, \dots, v_{N-2}$ free: $N-2$ complex numbers, hence

$$p = (x_1, \dots, x_{N-2},\, y_1, \dots, y_{N-2}) \in \mathbb{R}^{2N-4}.$$

Orientation is fixed by requiring positive (counter-clockwise) area, which excludes the mirror image; the mirror has the same spectrum, so nothing is lost.

Given p, let $C = P - \sum_{j=1}^{N-2} |v_{j+1} - v_j|$ (with $v_{N-1} = 0$) be the perimeter left for the two closing edges $0 \to v_0$ and $v_0 \to v_1$. Writing $v_0 = a > 0$, the constraint $|a - v_1| + a = C$ gives

$$a = \frac{C^2 - x_1^2 - y_1^2}{2\,(C - x_1)}, \tag{2}$$

real and positive exactly when $C > |v_1|$, i.e. when the two closing edges can reach. Outside that region no polygon exists; the admissible set is therefore a proper subset of $\mathbb{R}^{2N-4}$, which an unconstrained line search will leave.

2.1 The Jacobian, implicitly

We need $\partial v/\partial p \in \mathbb{R}^{2N\times(2N-4)}$. Differentiating (2) directly is possible and error-prone. Instead differentiate the constraint $\mathrm{per}(v) = P$ implicitly. With $u_k = (v_k - v_{k-1})/|v_k - v_{k-1}|$ the unit edge vectors,

$$\frac{\partial\,\mathrm{per}}{\partial v_k} = u_k - u_{k+1}, \tag{3}$$

and since x_j, y_j for $j = 1, \dots, N-2$ *are* the parameters while v_{N-1} and $\Im v_0$ are pinned, the only nontrivial row is

$$\frac{\partial x_0}{\partial p} = -\frac{1}{\partial\,\mathrm{per}\,/\partial x_0}\left(\frac{\partial\,\mathrm{per}}{\partial x_1}, \dots, \frac{\partial\,\mathrm{per}}{\partial y_{N-2}}\right). \tag{4}$$

Every column of the resulting Jacobian is tangent to $\{\mathrm{per} = P\}$, which is what lets an optimizer move without a multiplier.

One direction only. The map $p \mapsto v$ is smooth. Its inverse — putting a given polygon into the gauge — is *not*: it selects the longest side and normalizes orientation, and both choices jump when sides tie. A regular N-gon ties everywhere, and regular N-gons are what these problems converge to, so the canonical map has a branch cut sitting exactly on the target. Use it to build an initial p; never inside a loop.

3 The shape derivative of a simple eigenvalue

Let V be a velocity field on $\partial\Omega$ and Ω_t the domain transported by it. For a *simple* λ with $\|u\|_{L^2(\Omega)} = 1$, the Hadamard formula gives

$$\left.\frac{\mathrm{d}\lambda}{\mathrm{d}t}\right|_{t=0} = -\int_{\partial\Omega} \left(\frac{\partial u}{\partial n}\right)^2 (V\cdot n)\,\mathrm{d}s, \tag{5}$$

n the outward normal. Only the normal component enters: sliding the boundary along itself does not move the domain.

3.1 The velocity of a vertex

For a polygon parametrized by its vertices, the design variable *is* a vertex. Moving v_j by δ while holding the others fixed displaces the two incident edges linearly. In normalized arclength $\tau \in [0,1]$ along segment j (running $v_j \to v_{j+1}$),

$$\delta p(\tau) = \begin{cases} \tau\,\delta & \text{on segment } j-1 \quad (v_j \text{ is its endpoint}),\\ (1-\tau)\,\delta & \text{on segment } j \quad (v_j \text{ is its start}),\\ 0 & \text{otherwise,}\end{cases} \tag{6}$$

and $V\cdot n = \Re(\overline{n}\,\delta p)$. Both edges remain straight, so (6) is exact rather than a linearization.

One solve buys the whole gradient. The Cauchy data $\partial u/\partial n$ is computed once; each of the $2N$ partial derivatives is then (5) against a different $V\cdot n$, i.e. a weighted dot product. So $\nabla_p\lambda$ costs about one eigenvalue evaluation, not $2N$ of them — the reason to use Hadamard rather than differencing the solver.

A quadrature trap. A vertex perturbation *moves a corner*, and near a corner of interior angle ω with $\nu = \pi/\omega$ one has $(\partial u/\partial n)^2 \sim r^{2\nu-2}$, while (6) contributes $V\cdot n \sim r$. The corner-adapted quadrature that normalizes eigenfunctions is built for the eigenfunction's own exponent family and does *not* integrate that weight exactly. Measured on an L-shape's reentrant vertex against a five-point central difference: 6.1 digits with the normalization node set, 9.3 with the one built for integer powers, both answers looking entirely plausible. For $\nu > 1$ the integrand vanishes at the corner and the distinction is invisible — which is why the trap only appears once a reentrant corner becomes a design variable.

4 Multiple eigenvalues

Formula (5) requires u, and at a multiple eigenvalue there is no distinguished u. Let λ have multiplicity m with orthonormal eigenfunctions $u_1, \ldots, u_m$ spanning the eigenspace, and define

the symmetric $m \times m$ matrix

$$M(V)_{ij} \;=\; -\int_{\partial\Omega} \frac{\partial u_i}{\partial n}\,\frac{\partial u_j}{\partial n}\,(V\cdot n)\,\mathrm{d}s. \tag{7}$$

Then the m eigenvalue branches emerging from λ have one-sided derivatives equal to the *eigenvalues of* $M(V)$, ordered accordingly. Two consequences organize everything below.

1. $V \mapsto M(V)$ is **linear**. So along a parameter direction d, $M(d) = \sum_i d_i M^{(i)}$ is an affine matrix function, and the local model of an ordered eigenvalue is an ordered eigenvalue of an affine matrix function — the best-understood case in nonsmooth optimization.

2. Only the **spectrum** of M is meaningful. The basis $\{u_i\}$ is an arbitrary orthonormal basis of the eigenspace and rotates under perturbation, so individual entries carry no information. On the unit square's double $\lambda_2 = \lambda_3 = 5\pi^2$, an edge translation gives entries $-50.4, -48.3$ with off-diagonal -29.6, against eigenvalues -19.7 and -79.0: nothing about the entries resembles the answer.

Symmetric functions stay smooth. If f is symmetric in the cluster then $\sum_j f(\lambda_j) = \operatorname{tr} f(\Lambda + M)$ is a polynomial in the entries of M, hence smooth in the shape. In particular $\lambda_2 + \lambda_3$ has an honest gradient where λ_2 and λ_3 separately do not. Rephrasing an objective over a whole cluster is often cheaper than handling the nonsmoothness.

4.1 Near-degenerate clusters, and a basis trap

In practice a cluster is ε-active: its members are near-equal, not equal. If the cluster eigenvalues are $\lambda_\ell, \ldots, \lambda_{\ell+m-1}$ then the correct first-order model is the spectrum of

$$\Lambda + M(d), \qquad \Lambda = \operatorname{diag}(\lambda_\ell - \lambda_k, \ldots, \lambda_{\ell+m-1} - \lambda_k), \tag{8}$$

not of $M(d)$ alone. Dropping Λ makes the existing splitting appear free and inflates the predicted gain.

But Λ is diagonal only in the basis that diagonalizes the unperturbed cluster. A Löwdin orthonormalization of the eigenspace does not; adding Λ to an M built in that basis sums two matrices in different bases and yields a model whose predicted *direction* is wrong. The symptom is unmistakable once looked for: the trust-region ratio

$$\rho = \frac{F(p+d) - F(p)}{\text{predicted gain}}$$

stays *negative* as the radius shrinks ($-0.24, -0.49, -0.65, -0.75, -0.81, -1.23$ measured) where a correct first-order model must give $\rho \to 1$. The remedy is to build M from the individual eigenfunctions when the splitting is numerically resolvable, and to fall back to the cluster basis with $\Lambda = 0$ when it is not — there Λ vanishes and any orthonormal basis serves.

5 Differentiating the ratio

With $F = \lambda_k/\lambda_1$ and $g_1 = \nabla_p \lambda_1$ (which exists, by Fact 1), the directional derivative in direction d is

$$\mathrm{d}F(d) \;=\; \frac{\sigma_j(\Lambda + M(d))\,\lambda_1 - \lambda_k\,(g_1\cdot d)}{\lambda_1^2}, \tag{9}$$

where σ_j is the j-th ordered eigenvalue and j is the position of λ_k within its cluster. When the cluster is a singleton, σ_0 is linear and (9) is an ordinary gradient.

5.1 The dichotomy that decides the algorithm

$\lambda_{\min}$ of an affine matrix function is **concave**; $\lambda_{\max}$ is **convex**. Hence

position of λ_k	model of $\mathrm{d}F$	maximizing it
lowest of its cluster	concave	concave program: one optimum, tractable
highest of its cluster	convex	maximum on the boundary; nonconvex in general
simple	linear	ordinary steepest ascent

At regular N-gons the pattern (measured) is: λ_1 simple, then alternating doubles, with λ_2 always the *lowest* member of a double. So $k = 2$ — and $k = 4$ for $N \geq 5$ — lands in the tractable column, while $k = 3$ (and $k = 5$ for $N \geq 5$) does not.

6 The trust-region subproblems

At each iterate we maximize the model (9) over $\|d\| \leq \Delta$.

6.1 Concave case: λ_k lowest

Introducing t for $\lambda_{\min}$,

$$\max_{d,\,t} \quad t\lambda_1 - \lambda_k(g_1 \cdot d) \qquad \text{s.t.} \qquad \Lambda + M(d) \succeq tI, \quad \|d\| \leq \Delta, \tag{10}$$

a semidefinite program. For $m = 2$ the matrix inequality is two scalar conditions,

$$\operatorname{tr}\left(\Lambda + M(d) - tI\right) \geq 0, \qquad \det\left(\Lambda + M(d) - tI\right) \geq 0,$$

so (10) is a small smooth NLP and no SDP solver is required. Since the dihedral symmetry D_N produces $m = 2$ generically, the cheap case is also the common one.

6.2 Convex case: λ_k highest

Here the subproblem maximizes a convex function over a ball. For $m = 2$ it nonetheless reduces exactly to a one-dimensional search. Write $M(d) = \begin{pmatrix} \alpha & \beta \\ \beta & \gamma \end{pmatrix}$ with $\alpha = A \cdot d$, $\beta = B \cdot d$, $\gamma = C \cdot d$ linear in d. Then

$$\lambda_{\max}(M(d)) = \underbrace{\tfrac{\alpha+\gamma}{2}}_{\ell \cdot d} + \Big\| \underbrace{\left(\tfrac{\alpha-\gamma}{2},\ \beta\right)}_{Ld} \Big\| \ = \ \ell \cdot d + \|Ld\|, \qquad L : \mathbb{R}^{n_p} \to \mathbb{R}^2. \tag{11}$$

Using $\|w\| = \max_{\|u\| \leq 1} u^\top w$ and exchanging the maxima,

$$\max_{\|d\| \leq \Delta} \Big[\ell \cdot d + \|Ld\| \Big] \ = \ \max_{\|u\| \leq 1} \ \Delta \, \|\ell + L^\top u\|. \tag{12}$$

Since $u \in \mathbb{R}^2$, (12) is a maximization over the unit circle: a one-dimensional search, exact, with the optimal d recovered from the maximizing u. The convex case is therefore not the research obstacle it first appears to be, provided $m = 2$.

6.3 Criticality is gain per unit radius

The optimal value of (10) scales with Δ. Testing it directly declares optimality whenever the radius has shrunk — observed at a point whose directional derivative was still 1.4. The stopping test must be

$$\text{criticality} \;=\; \frac{\text{optimal model gain}}{\Delta},$$

which is the largest achievable directional derivative and vanishes only at a first-order critical point.

7 First-order optimality, and a condition that is vacuous

Take $k = 2$ at a regular N-gon. Two facts hold there to machine precision, both measured:

1. $g_1 = 0$, because the regular N-gon *minimizes* λ_1 at fixed perimeter (Pólya–Szegő for $N = 3, 4$);
2. $\operatorname{tr} M(d) = 0$ for all d, i.e. $\lambda_2 + \lambda_3$ is stationary.

With $g_1 = 0$, (9) collapses to $\mathrm{d}F(d) = \lambda_{\min}(M(d))/\lambda_1$, so a local maximum requires $\lambda_{\min}(M(d)) \le 0$ for all d. Because $M(-d) = -M(d)$ this is equivalent to $M(d)$ never being definite, and for $m = 2$ exactly to $\det M(d) \le 0$.

Now use the second fact. Traceless means $\gamma = -\alpha$, i.e. $C = -A$, so

$$\det M(d) = (A{\cdot}d)(C{\cdot}d) - (B{\cdot}d)^2 = -(A{\cdot}d)^2 - (B{\cdot}d)^2 \;\le\; 0 \qquad \text{identically.} \tag{13}$$

The quadratic form is $Q = -AA^\top - BB^\top$: negative semidefinite by construction, of rank at most 2 whatever n_p is (measured rank exactly 2 against $n_p = 4, 6, 8, 10$).

So the first-order condition is satisfied automatically and certifies nothing. It cannot distinguish a maximum from anything else. This is worth stating plainly because an earlier version of these notes reported the determinant test as a certificate on the strength of margins of order 10^{-13} — which were not small, but structurally zero.

The content lies in the $n_p - 2$ dimensional null space $\{d : A{\cdot}d = B{\cdot}d = 0\}$, where $\mathrm{d}F = 0$ and the verdict is second order. Measured there, F falls quadratically along every sampled direction. That is evidence, not proof. The exception is $N = 3$: then $n_p = 2$ equals $\operatorname{rank} Q$, the null space is trivial, $Q \prec 0$ strictly, and the equilateral triangle is a genuine first-order strict local maximizer.

8 Invariants used as checks

Because a wrong shape derivative is finite and plausible rather than catastrophic, we check against statements that are true independently of the code.

invariant	what it caught
Fact 2, $\lambda_2/\lambda_1 \le 2.5387340$	a reported 2.6936: tracked eigenvalues were returned unsorted after λ_2 and λ_3 crossed, so the objective read the third mode as the second. Also a reported 2.5400: the objective was stale relative to the eigenvalues it was supposed to describe.
Pólya–Szegő, regular N-gon minimizes λ_1	a run converging 2.8×10^{-3} *below* $2\pi^2$ for the square, which is impossible; the eigenvalue search had accepted a point whose tension was 2×10^{-4}.
$\rho \to 1$ as $\Delta \to 0$	the basis mismatch of §4.1.
Rigid motions leave λ fixed	sign and bookkeeping errors in the assembled $\nabla_v \lambda$, which must sum to zero over all vertices in each coordinate.

9 What is not handled

- **Clusters with $m > 2$.** Both (10) and (12) are written for 2×2 blocks. General m needs an SDP solver for the concave case; the convex case is genuinely harder, since (11) is special to $m = 2$.
- **Global optimality.** Everything here is local. Whether the regular N-gon maximizes λ_2/λ_1 among N-gons is open; we have local evidence only.
- **Corner-moving velocities at a reentrant corner in the certified regime.** The quadrature caveat above is handled by a separate node set, but the interaction with very sharp corners ($\nu \ll 1$) is untested.

Pointers

§2	`douse/polygon.py`
(5)–(6)	`douse/derivative.py`; the Hadamard node set is `lappy`'s `MPSEigensolver.hadamard_quad`
§4–§4.1	`douse/derivative.py` (`cluster_matrices`), `douse/cluster_opt.py` (`local_model`)
(10)	`douse/cluster_opt.py` (`trust_region_step`)
§7	`douse/certify.py`